Prolog Programs & Outputs (Q26 - Q33)

SWI-Prolog code with sample queries and output

26. Write a Prolog Program for fruit and its color using Back Tracking.

Aim: Facts store fruit-color pairs. When a query has several answers, Prolog backtracks to the next matching fact (typing ; in the terminal). The rule show_all/0 forces backtracking with fail to print every pair.

Program

```
% Facts: fruit_color(Fruit, Color)
fruit_color(apple,  red).
fruit_color(banana, yellow).
fruit_color(grape,  purple).
fruit_color(orange, orange).
fruit_color(mango,  yellow).
fruit_color(cherry, red).

% Print all fruits and colors using backtracking
show_all :-
    fruit_color(F, C),
    write(F), write(' is '), write(C), nl,
    fail.            % force backtracking to the next fact
show_all.            % succeed after all facts are exhausted

% Print all fruits having a given color
fruits_of_color(C) :-
    fruit_color(F, C),
    write(F), nl,
    fail.
fruits_of_color(_).
```

Output

```
?- fruit_color(apple, X).
X = red.

?- fruit_color(F, yellow).
F = banana ;
F = mango.

?- show_all.
apple is red
banana is yellow
grape is purple
orange is orange
mango is yellow
cherry is red
true.

?- fruits_of_color(red).
apple
cherry
true.
```

27. Write a Prolog Program to implement Best First Search algorithm

Aim: Best First Search always expands the open path whose last node has the smallest heuristic value h(N). Graph: s->a,b ; a->c,d ; b->e,f ; e->g ; f->g. Goal is g.

Program

```
% Graph edges
edge(s, a).  edge(s, b).
edge(a, c).  edge(a, d).
edge(b, e).  edge(b, f).
edge(e, g).  edge(f, g).

% Heuristic values (estimated cost to goal g)
h(s, 10). h(a, 7). h(b, 5). h(c, 9).
h(d, 6).  h(e, 3). h(f, 4). h(g, 0).

% best_first(Start, Goal, Path)
best_first(Start, Goal, Path) :-
    bfs([[Start]], Goal, RevPath),
    reverse(RevPath, Path).

% Goal reached
bfs([[Goal|Rest]|_], Goal, [Goal|Rest]) :- !.

% Expand the best (first) path, add children, re-sort by heuristic
bfs([[N|Rest]|Others], Goal, Path) :-
    format('Visiting: ~w~n', [N]),
    findall([M, N|Rest],
            ( edge(N, M), \+ member(M, [N|Rest]) ),
            Children),
    append(Children, Others, Open),
    sort_by_h(Open, Sorted),
    bfs(Sorted, Goal, Path).

% Sort the open list of paths by h-value of their head node
sort_by_h(Paths, Sorted) :-
    findall(H-P, ( member(P, Paths), P = [N|_], h(N, H) ), Pairs),
    keysort(Pairs, SortedPairs),
    findall(P, member(_-P, SortedPairs), Sorted).
```

Output

```
?- best_first(s, g, Path).
Visiting: s
Visiting: b
Visiting: e
Path = [s, b, e, g].
```

28. Write the prolog program for Medical Diagnosis

Aim: A rule-based expert system: each disease is defined by the symptoms it requires. The program matches the symptoms recorded for a patient against the disease rules.

Program

```
% Patient symptoms: symptom(Patient, Symptom)
symptom(john, fever).
symptom(john, cough).
symptom(john, body_ache).

symptom(mary, headache).
symptom(mary, nausea).
symptom(mary, sensitivity_to_light).

symptom(raj, sneezing).
symptom(raj, runny_nose).

symptom(sita, fever).
symptom(sita, rash).

% Diagnosis rules: disease(Patient, Disease)
disease(P, flu)      :- symptom(P, fever), symptom(P, cough),
                        symptom(P, body_ache).
disease(P, migraine) :- symptom(P, headache), symptom(P, nausea),
                        symptom(P, sensitivity_to_light).
disease(P, cold)     :- symptom(P, sneezing), symptom(P, runny_nose).
disease(P, measles)  :- symptom(P, fever), symptom(P, rash).

% Print every possible diagnosis for a patient
diagnose(P) :-
    disease(P, D),
    format('~w may be suffering from ~w~n', [P, D]),
    fail.
diagnose(_).
```

Output

```
?- disease(mary, D).
D = migraine.

?- diagnose(john).
john may be suffering from flu
true.

?- diagnose(raj).
raj may be suffering from cold
true.

?- diagnose(sita).
sita may be suffering from measles
true.

?- disease(raj, flu).
false.
```

29. Write a Prolog Program for forward Chaining. Incorporate required queries.

Aim: Forward chaining starts from the known facts and repeatedly applies rules whose conditions are all true, adding new facts until nothing more can be derived (data-driven reasoning).

Program

```
:- dynamic fact/1.

% Initial known facts
fact(has_feathers).
fact(lays_eggs).
fact(can_fly).

% Rules: rule(ListOfConditions, Conclusion)
rule([has_feathers],        bird).
rule([lays_eggs, bird],     oviparous_animal).
rule([bird, can_fly],       flying_bird).

% Check that all conditions are known facts
all_true([]).
all_true([H|T]) :- fact(H), all_true(T).

% Forward chaining: fire a rule, add its conclusion, repeat
forward :-
    rule(Conds, Concl),
    \+ fact(Concl),
    all_true(Conds),
    assertz(fact(Concl)),
    format('Derived: ~w~n', [Concl]),
    !,
    forward.
forward.
```

Output

```
?- forward.
Derived: bird
Derived: oviparous_animal
Derived: flying_bird
true.

?- fact(flying_bird).
true.

?- fact(bird).
true.

?- fact(fish).
false.
```

30. Write a Prolog Program for backward Chaining. Incorporate required queries.

Aim: Backward chaining starts from the goal and works backwards: a goal is proved if it is a known fact, or if there is a rule whose conditions can all be proved (goal-driven reasoning).

Program

```
% Known facts
fact(has_feathers).
fact(lays_eggs).
fact(can_fly).

% Rules: rule(Goal, ListOfConditions)
rule(bird,             [has_feathers]).
rule(oviparous_animal, [lays_eggs, bird]).
rule(flying_bird,      [bird, can_fly]).

% prove(Goal): true if Goal is a fact or derivable by a rule
prove(Goal) :- fact(Goal).
prove(Goal) :-
    rule(Goal, Conditions),
    prove_all(Conditions).

prove_all([]).
prove_all([H|T]) :- prove(H), prove_all(T).

% Same, but prints the reasoning
explain(Goal) :-
    prove(Goal),
    format('~w is TRUE~n', [Goal]), !.
explain(Goal) :-
    format('~w cannot be proved~n', [Goal]).
```

Output

```
?- prove(flying_bird).
true.

?- prove(oviparous_animal).
true.

?- prove(fish).
false.

?- explain(bird).
bird is TRUE
true.

?- explain(fish).
fish cannot be proved
true.
```

31. Create a Web Blog using Word press to demonstrate Anchor Tag, Title Tag, etc.

Steps

```
1. Log in to the WordPress dashboard (yoursite.com/wp-admin).
2. Go to Posts -> Add New. Enter the post title (this becomes the Title tag of the post page).
3. Click the "+" button and add a "Custom HTML" block.
4. Paste the HTML code given below into the block and click Preview / Publish.
5. Open the published post in the browser to view the output.
```

Code (HTML)

```
<h2>My First Blog Post</h2>

<!-- Title attribute (shows a tooltip on hover) -->
<p title="This is a paragraph with a title attribute">
  Hover over this paragraph to see the title tooltip.
</p>

<!-- Anchor tag: external link opening in a new tab -->
<p>Visit <a href="https://www.wordpress.org" target="_blank"
   title="Go to WordPress.org">WordPress.org</a></p>

<!-- Anchor tag: link to a page of the same site -->
<p><a href="/about" title="About this blog">About Me</a></p>

<!-- Anchor tag: internal link (bookmark) within the same page -->
<p><a href="#contact">Jump to Contact section</a></p>

<!-- Email link -->
<p><a href="mailto:student@example.com">Email me</a></p>

<!-- Other common tags -->
<h3>Other Tags</h3>
<p><b>Bold</b>, <i>Italic</i>, <u>Underline</u></p>
<ul>
  <li>HTML</li>
  <li>CSS</li>
</ul>
<img src="photo.jpg" alt="Sample image" title="My Photo" width="200">

<h3 id="contact">Contact</h3>
<p>Write to us at the e-mail address above.</p>

<!-- The <title> tag in a theme's header.php (page title in browser tab) -->
<title><?php wp_title('|', true, 'right'); bloginfo('name'); ?></title>
```

Output (as displayed in the blog)

```
Browser tab title : My First Blog Post | My Blog

My First Blog Post
Hover over this paragraph to see the title tooltip.
        (tooltip on hover: "This is a paragraph with a title attribute")
Visit WordPress.org            <- clickable link, opens in a new tab
About Me                       <- clickable link to the About page
Jump to Contact section        <- clickable, scrolls to the Contact heading
Email me                       <- clickable, opens the mail client

Other Tags
Bold, Italic, Underline
 * HTML
 * CSS
[ image: photo.jpg ]  (tooltip: "My Photo")

Contact
Write to us at the e-mail address above.
```

32. Write a Prolog program to implement pattern matching

Aim: Prolog matches patterns using unification. The program shows structure matching, list matching, and searching for a sub-pattern inside a list or an atom.

Program

```
% Facts used for structure pattern matching
book(title(prolog), author(bratko),  year(2012)).
book(title(ai),     author(russell), year(2020)).

% pattern_match(Pattern, List): Pattern occurs as a sub-list of List
pattern_match(Pattern, List) :-
    once(( append(_, Rest, List),
           append(Pattern, _, Rest) )).

% atom_match(Pattern, Text): Pattern occurs inside the atom Text
atom_match(Pattern, Text) :-
    atom_chars(Pattern, P),
    atom_chars(Text, T),
    pattern_match(P, T).
```

Output

```
?- point(X, 3) = point(2, Y).
X = 2,
Y = 3.

?- [H|T] = [a, b, c].
H = a,
T = [b, c].

?- book(title(T), author(russell), year(Y)).
T = ai,
Y = 2020.

?- pattern_match([b, c], [a, b, c, d]).
true.

?- pattern_match([x, y], [a, b, c, d]).
false.

?- atom_match(log, prolog).
true.

?- atom_match(cat, prolog).
false.
```

33. Write a Prolog program to find the number of vowels

Aim: The atom is converted into a list of characters. The list is traversed recursively and the counter is increased whenever the character is a vowel (upper or lower case).

Program

```
% Vowel definition (both cases)
vowel(X) :- member(X, [a,e,i,o,u,'A','E','I','O','U']).

% count_vowels(Atom, Count)
count_vowels(Atom, Count) :-
    atom_chars(Atom, Chars),
    count(Chars, Count).

count([], 0).
count([H|T], C) :-
    vowel(H), !,
    count(T, C1),
    C is C1 + 1.
count([_|T], C) :-
    count(T, C).

% Print the result
vowels(Atom) :-
    count_vowels(Atom, C),
    format('Number of vowels in ~w = ~d~n', [Atom, C]).
```

Output

```
?- count_vowels(hello, C).
C = 2.

?- count_vowels(education, C).
C = 5.

?- vowels('Prolog Programming').
Number of vowels in Prolog Programming = 5
true.

?- vowels('Artificial Intelligence').
Number of vowels in Artificial Intelligence = 10
true.

?- vowels(rhythm).
Number of vowels in rhythm = 0
true.
```